

Capstone Workshop

Audit Trail & Dashboard Paging

A Two-Worktree, Multi-Agent Development Workflow — ABC Healthcare Prior Authorization App

This handout gives you the context, the exact problem you're solving, and a trail of clues to get you there — not the code. You will use Git worktrees, Graphify (code graph), the GitHub MCP server, one additional MCP server of your choice, Claude Code subagents (foreground and background), skills, chained custom commands, and session cost tracking.

*Note: Member Eligibility Verification is **out of scope** for this capstone — that feature is being implemented separately through the SDD pipeline with its own guardrails, evals, and golden dataset.*

1. Current Application Gaps

Both gaps below were verified directly against the repository — they are not hypothetical. Together they make the Prior Authorization Dashboard impossible to fully trust or fully use.

GAP 1 — No Status Audit Trail

File: `backend/PriorAuth.API/Controllers/AuthorizationsController.cs` —
`UpdateStatus(int id, UpdateStatusRequest request)`

The PATCH endpoint overwrites `auth.Status` directly. No row, table, or log entry is created recording the previous status, the new status, or when the change happened.

Verified absence: searching `database/init.sql` and `Models/Entities.cs` for "history" or "audit" returns zero matches. No such table exists anywhere in the schema.

The only validation performed is that the new status is one of five known strings. Nothing prevents an illegal transition — e.g. `DENIED` → `APPROVED` is silently accepted today.

Business impact: in a healthcare prior-authorization workflow, knowing who changed a request's status, when, and from what is a baseline operational expectation. Right now, once a status changes, the previous value is gone — permanently.

GAP 2 — Dashboard Ignores Its Own Backend's Pagination

File: `backend/PriorAuth.API/Controllers/AuthorizationsController.cs` —
already implements paging server-side:

```
GetAll(status, page = 1, pageSize = 20) → .Skip().Take()
```

File: `frontend/src/api/client.ts` — never sends either parameter:

```
authorizations.getAll(status) // page, pageSize omitted
```

Result: `DashboardPage.tsx` silently caps at whatever the backend's default page is, with no control to go further and no indication that more records exist beyond it.

2. Problem Statement

The Prior Authorization Dashboard cannot currently be trusted for two independent operational reasons:

- Once a reviewer changes a request's status, there is no record anywhere of what it was before, who changed it, or when — the system is not auditable.
- The dashboard can only ever display one page of results, with a pagination capability already built on the backend and simply never wired up.

Your capstone is to close both gaps as a single coordinated feature, built the way a real engineering team would: two parallel workstreams (backend, frontend), integrated through Git worktrees and GitHub, with an AI-agent workflow doing the actual implementation work under your direction.

3. Your Task

3.1 — Backend Worktree: Status Audit Trail

- [] Add a new table that captures status-change history for every Authorization (previous status, new status, changed-at timestamp, at minimum).
- [] Every call to `PATCH /api/authorizations/{id}/status` must write a history row before it returns — no exceptions.
- [] Reject illegal status transitions with a clear error instead of silently accepting any value in the fixed list.
- [] Add `GET /api/authorizations/{id}/history` returning the full history, in order.

3.2 — Frontend Worktree: Pagination + History UI

- [] Add page navigation to the Dashboard, wired to the backend's existing `page/pageSize` parameters.
- [] Add a Status History panel to the Authorization Detail page, populated from the new backend endpoint.
- [] Reuse the existing `STATUS_COLORS` / `STATUS_BG` constants for any status shown in history — do not invent new colors.

3.3 — Required Workflow Elements

Across both worktrees, your process (not just your output) must include all of the following at least once:

- [] Two separate Git worktrees, one branch each.
- [] A Graphify graph built/queried before any file is edited in either worktree.
- [] At least one custom command that chains multiple subagents end to end.
- [] At least one subagent invoked as a skill.
- [] At least one subagent explicitly run in the background.
- [] The GitHub MCP server, for issue intake and opening both PRs.
- [] One additional MCP server of your choice (see Section 4).
- [] A `/cost` or `/usage` check reported at the end of each worktree's work.

3.4 — Out of Scope

Do not build these in this capstone, even though they are real, known gaps in the app:

- [] Member Eligibility Verification — owned by the separate SDD pipeline.
 - [] Authentication / authorization on the API.
 - [] The PHI-in-logging exposure or the duplicate reference-number risk found in earlier sessions.
-

4. Ground Rules

Two worktrees, one repo:

```
git worktree add ../wt-backend-audit -b feature/status-audit-trail
git worktree add ../wt-frontend-ui -b feature/dashboard-pagination-history
```

Pick your second MCP server:

Both options below are real, well-known, and safe (no code execution against untrusted input) — pick whichever fits the moment you need it:

- **Context7** — documentation-only lookup. Good for pulling current EF Core migration syntax before generating one, instead of trusting possibly-stale training data.
- **Playwright MCP** (official, Microsoft) — drives a real browser. Good for verifying the pager and the history panel actually render and respond, before either PR opens.

Merge order is not arbitrary:

The backend worktree owns the `/history` contract. It must merge before the frontend's history panel switches from a stub to live data.

5. Definition of Done

- [] History table exists and is populated on every status change.
- [] Illegal transitions are rejected with an error, not silently accepted.
- [] `GET .../history` returns ordered history.
- [] Dashboard pagination works end to end against real data.
- [] Status History panel shows real (not stubbed) data after merge.
- [] Both PRs opened and linked to one shared GitHub issue.
- [] At least one skill, one background subagent, one chained command, and one Graphify query can each be pointed to in your session history.
- [] `/cost` or `/usage` reported for each worktree.

6. Solution Approach — Clues

These are clues, not code. Each one points you at a decision or a tool call — the implementation is yours. Work them roughly in order within each track; the SETUP clues apply to both worktrees.

SETUP

CLUE 1 — Two rooms, one house

Git already has a command that gives you two independent working directories from the same repository, without cloning it twice. You've seen it in Section 4.

CLUE 2 — Map before you move

Before you edit a single file in either worktree, ask the codebase what's already there instead of assuming. There's a command that turns "grep and hope" into a queryable map.

```
graphify install (once, if not already set up)
/graphify . (run inside each worktree)
```

BACKEND TRACK

CLUE 3 — Know who's watching before you touch it

You're about to change how a field is written. Something else in the controller might already depend on the old behavior — ask the graph before you assume it's isolated.

```
graphify query "what calls UpdateStatus in AuthorizationsController"
```

CLUE 4 — Five relationships already exist

Authorization already connects to Member, Provider, HealthPlan, Site, and its diagnosis/procedure tables. Your new history table needs to attach cleanly to that shape — see it before you design it.

```
graphify explain "Authorization"
```

CLUE 5 — Don't trust your memory of the API

EF Core's migration API surface shifts across versions, and training data goes stale. One of your two MCP options exists specifically to hand you current, correct syntax before you generate anything.

CLUE 6 — Two different jobs, one method

"Is this string one of five valid values" and "is this transition actually allowed from where the request currently is" are two different pieces of logic. Right now only the first one exists. The second one is a good candidate for its own skill.

CLUE 7 — You've made this mistake once already

Somewhere earlier in this app, verbose logging leaked patient data into plaintext logs. Your new history rows will contain patient-adjacent data too. Make sure whatever writes or logs them doesn't become the second instance of the same mistake.

CLUE 8 — Not everything has to wait its turn

Writing the migration and writing the transition-guard skill don't depend on each other's output to get started. One of those two doesn't need to run in the foreground while you watch it.

FRONTEND TRACK

CLUE 9 — The handoff breaks in exactly two files

The backend already accepts page and pageSize. Something between the Dashboard and the controller is dropping them on the floor. Find precisely where.

```
graphify path "DashboardPage" "AuthorizationsController.GetAll"
```

CLUE 10 — Changing a shared function's shape is only safe if...

...exactly one thing calls it. Confirm that before you touch its signature — if the answer isn't "just one caller," your change has a second, hidden target.

```
graphify query "what calls api.authorizations.getAll"
```

CLUE 11 — You don't need real data to start building

The History panel needs an agreed response shape, not a finished backend. Decide what that shape looks like and build the UI against it now — swap it for the real endpoint later, at merge time.

CLUE 12 — This app already has an opinion about status colors

One file already maps every status to a color, twice over. A new UI element that picks its own colors for the same five statuses is a smell — something in your workflow should catch it before merge.

CLUE 13 — "It compiled" is not "it works"

Before either PR opens, something should actually click the pager and open the history panel in a real browser. One of your two MCP options exists exactly for this.

MERGE & WRAP-UP

CLUE 14 — Order matters here, for real

One worktree owns a contract the other is depending on. Merge them in the wrong order and you're integrating the second one against a target that's still moving.

CLUE 15 — Your map just went stale

The graph you built in Clue 2 doesn't know about the table or the component you just added. There's a reason the tool supports updating a graph, not just building one once.

CLUE 16 — What did this actually cost you?

Two worktrees, two sessions, presumably two different bills. There's a one-word command that tells you exactly what each session cost — run it before you close either session, not after you've lost track of which one ran longer.

Good luck — and remember: the point of this capstone isn't just a merged PR. It's showing you actually understand which tool answers which question, and when to hand work to a subagent versus watch it yourself.